

Portfolio Web App Deployment in Kubernetes Using Minikube

Minikube Start

To begin setting up our development environment for the portfolio project, we initiated Minikube, a local Kubernetes cluster. First, ensure that Minikube is installed on your system. In the terminal, run the command `"minikube start"`. This command launches a single-node Kubernetes cluster on your local machine, allowing you to develop and test Kubernetes applications without needing a cloud provider. Once the cluster is up and running, you will see a confirmation message indicating that Minikube is ready to use. This setup is essential for deploying our backend services within a controlled environment, making it easier to test and manage application resources.

```
Jayasurya@jayasurya:~$ minikube start
minikube v1.33.1 on Ubuntu 22.04
Using the docker driver based on existing profile
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.44 ...
docker "minikube" container is missing, will recreate.
Creating docker container (CPUs=2, Memory=2200MB) ...
Preparing Kubernetes v1.30.0 on Docker 26.1.1 ...
  ■ Generating certificates and keys ...
  ■ Booting up control plane ...
  ■ Configuring RBAC rules ...
Configuring bridge CNI (Container Networking Interface) ...
Verifying Kubernetes components...
  ■ Using image gcr.io/k8s-minikube/storage-provisioner:v5
  ■ Using image docker.io/kubernetes/dashboard:v2.7.0
  ■ Using image docker.io/kubernetes/metrics-server:v1.0.8
Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

Enabled addons: storage-provisioner, default-storageclass, dashboard
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
Jayasurya@jayasurya:~$
```

Minikube Status

After starting Minikube, it's important to check the cluster's status. Use the command `"minikube status"` in your terminal. This will display the current state of the Minikube cluster, including whether it's running and the status of key components like the Kubernetes API server and kubelet. A successful status check confirms that the cluster is operational, allowing you to proceed with application deployment.

```
Jayasurya@jayasurya:~$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
Jayasurya@jayasurya:~$
```

Apply Deployment and Service

To deploy your application and expose it as a service in your Kubernetes cluster, use the “`kubectl apply`” command for both the deployment and the service configuration files. For example, executing “`kubectl apply -f deployment.yaml`” creates the deployment, while “`kubectl apply -f service.yaml`” exposes the application to the network. These commands ensure that your application is running in the cluster and accessible as specified in the YAML files.

```
Jayasurya@jayasurya:~/Downloads$ kubectl apply -f Portfolio-Deployment.yaml
deployment.apps/portfolio created
Jayasurya@jayasurya:~/Downloads$ kubectl apply -f Portfolio-service.yaml
service/portfolio-service created
```

Get Running Pods from Minikube

To view the status of the pods running in your Kubernetes cluster, use the command “`kubectl get pods`”. This command lists all the pods along with their current status, indicating whether they are running, pending, or in an error state. The output will display the pod names, ready status, restarts, and the age of each pod, providing essential information for monitoring your application's health. Checking the status of your pods is a crucial step after deployment, as it ensures that your application is functioning as expected.

```
Jayasurya@jayasurya:~/Downloads$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
portfolio-5ff6cb87c-gjbt4          1/1     Running   0           22s
portfolio-5ff6cb87c-qh8b9          1/1     Running   0           22s
```

Get Running Services

To check the status of the services in your Kubernetes cluster, use the command “`kubectl get svc`”. This command lists all the services along with their details, including the service name, type (e.g., ClusterIP, NodePort), cluster IP, external IP (if applicable), ports, and age. This information is vital for understanding how your application is exposed within the cluster and whether it's accessible to users. Monitoring services is essential to ensure that your application is properly configured and reachable.

```
Jayasurya@jayasurya:~/Downloads$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	23d
portfolio-service	NodePort	10.107.93.246	<none>	8080:32437/TCP	25s

Port Forwarding

Port forwarding in Kubernetes allows you to access a specific pod or service from your local machine by forwarding a port on your localhost to a port on the Kubernetes cluster. This is particularly useful for testing applications without exposing them externally. To set up port forwarding, use the command “`kubectl port-forward svc/<service-name> <local-port>:<service-port>`”.

```
Jayasurya@jayasurya:~/Downloads$ kubectl port-forward svc/portfolio-service 8080:8080
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

Accessing the Portfolio Service

To access the Portfolio application service running in your Minikube cluster, you can use the command `minikube service portfolio-service`. This command will automatically open your default web browser and direct it to the service's URL, providing a convenient way to interact with your application. It retrieves the external URL assigned to the service, allowing you to access the application as if it were hosted on a public server. This command is particularly useful for testing and development purposes, as it enables quick access to your deployed services without needing to configure port forwarding manually.

```
Jayasurya@jayasurya:~/Downloads$ minikube service portfolio-service
```

NAMESPACE	NAME	TARGET PORT	URL
default	portfolio-service	8080	http://192.168.58.2:32437

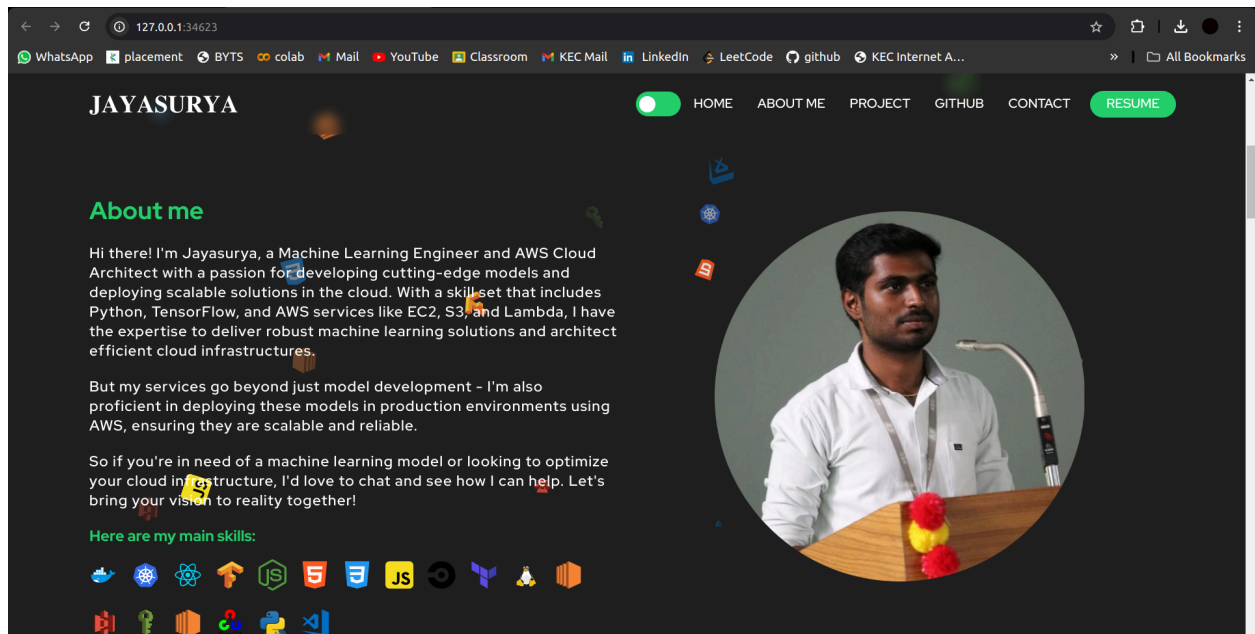
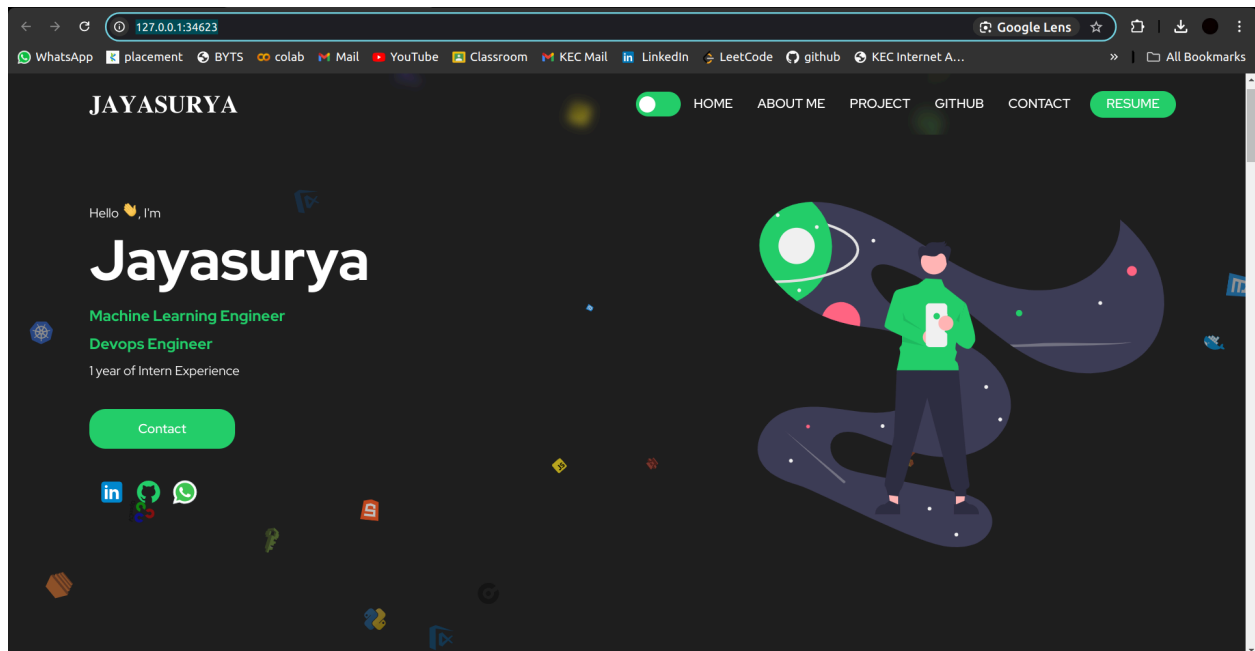
```
Starting tunnel for service portfolio-service.
```

NAMESPACE	NAME	TARGET PORT	URL
default	portfolio-service		http://127.0.0.1:34623

```
Opening service default/portfolio-service in default browser...
! Because you are using a Docker driver on linux, the terminal needs to be open to run it.
Opening in existing browser session.
```

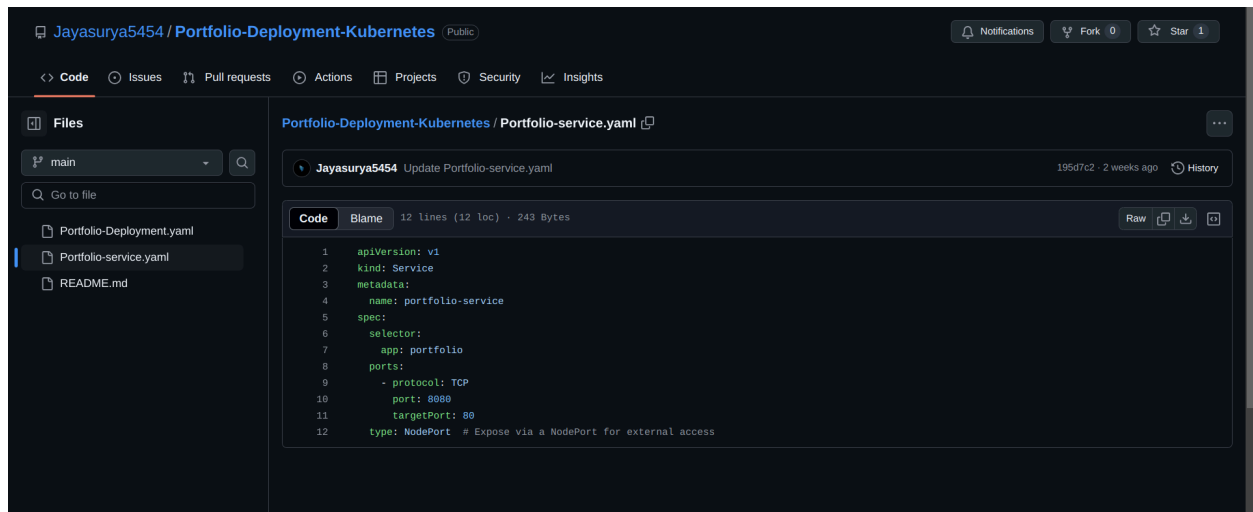
Application Running on Localhost

After successfully deploying your application in the Kubernetes cluster, you can access it through your local machine. If you have set up a service to expose your application, you can either use **port forwarding** or the **Minikube service** command to access the application.



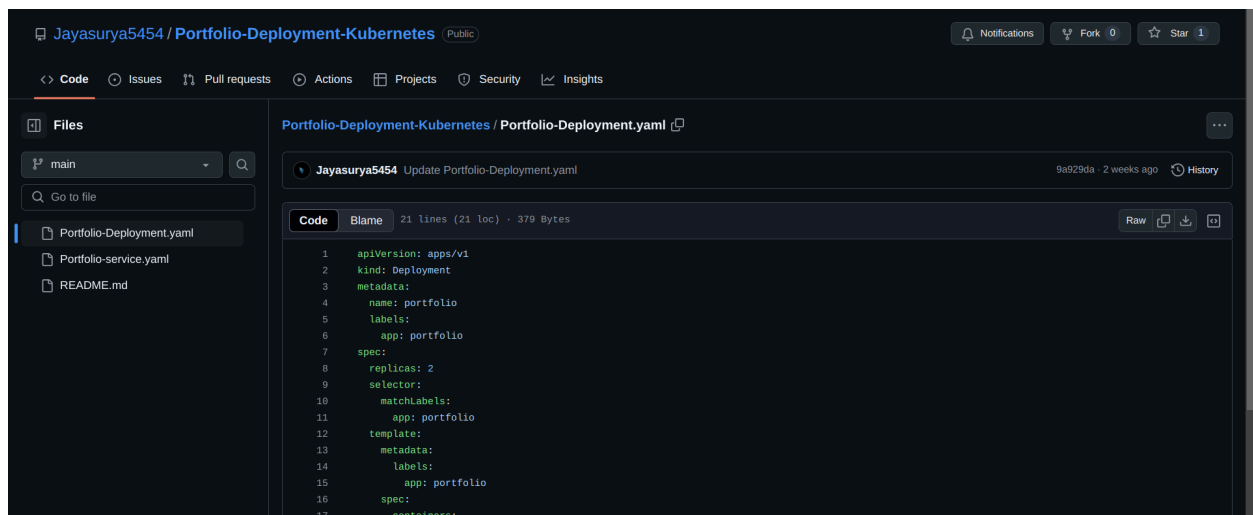
Deployment Codes in GitHub Repository

To manage Kubernetes deployment configurations and application code effectively, it's best practice to store them in a version control system like GitHub.



This screenshot shows the GitHub interface for the repository 'Jayasurya5454 / Portfolio-Deployment-Kubernetes'. The left sidebar displays the file structure with 'Portfolio-service.yaml' selected. The main content area shows the details of a commit titled 'Update Portfolio-service.yaml' by 'Jayasurya5454'. The code is displayed in a dark-themed editor with line numbers 1 through 12. The code defines a Kubernetes Service named 'portfolio-service' with a selector 'app: portfolio' and a port '8080'.

```
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: portfolio-service
5 spec:
6   selector:
7     app: portfolio
8   ports:
9     - protocol: TCP
10       port: 8080
11       targetPort: 80
12 type: NodePort # Expose via a NodePort for external access
```



This screenshot shows the GitHub interface for the repository 'Jayasurya5454 / Portfolio-Deployment-Kubernetes'. The left sidebar displays the file structure with 'Portfolio-Deployment.yaml' selected. The main content area shows the details of a commit titled 'Update Portfolio-Deployment.yaml' by 'Jayasurya5454'. The code is displayed in a dark-themed editor with line numbers 1 through 17. The code defines a Kubernetes Deployment named 'portfolio' with 2 replicas and a selector 'app: portfolio'.

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: portfolio
5   labels:
6     app: portfolio
7 spec:
8   replicas: 2
9   selector:
10     matchLabels:
11       app: portfolio
12   template:
13     metadata:
14       labels:
15         app: portfolio
16   spec:
17     containers:
```

Accessing the Minikube Dashboard

To view the Kubernetes dashboard in Minikube, you can use the command “`minikube dashboard --url`”. The dashboard allows you to visualize resources like pods, deployments, services, and more, making it easier to monitor and manage your applications. Upon executing the command, your default web browser will open with the dashboard URL, providing an intuitive way to interact with your cluster without relying solely on the command line.

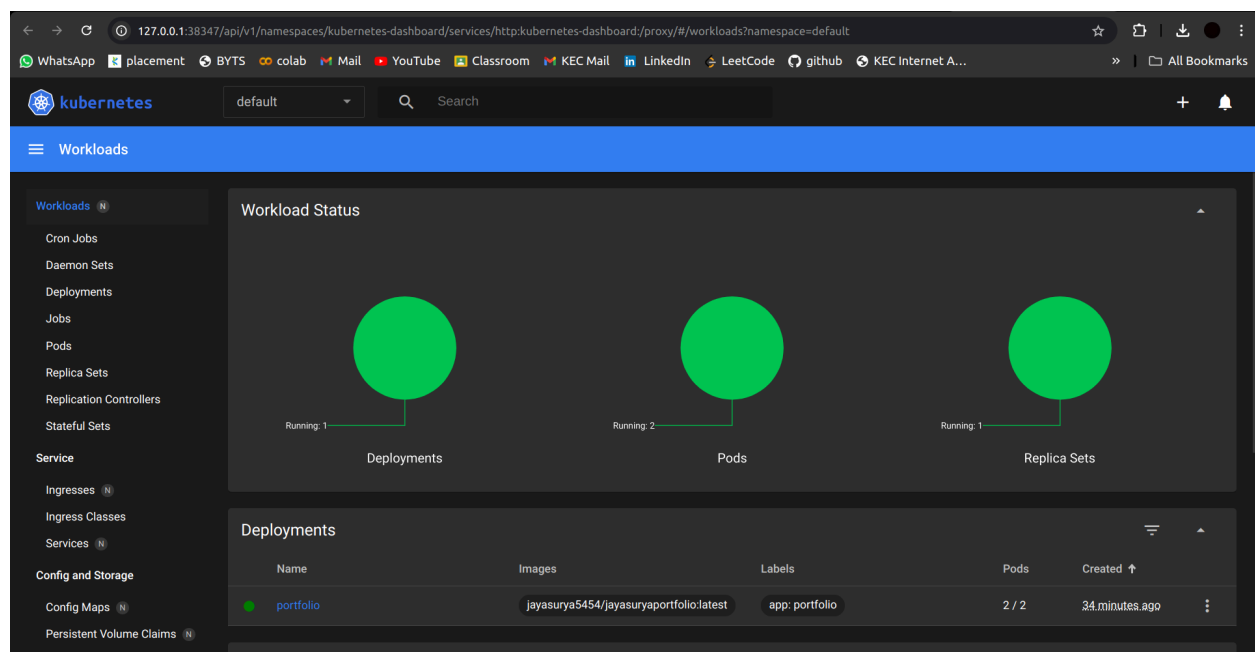
```
Jayasurya@jayasurya:~$ minikube dashboard --url
🔧 Enabling dashboard ...
  ■ Using image docker.io/kubernetes/metrics-scraper:v1.0.8
  ■ Using image docker.io/kubernetes/dashboard:v2.7.0
💡 Some dashboard features require the metrics-server addon. To enable all features please run:

    minikube addons enable metrics-server

🤖 Verifying dashboard health ...
🚀 Launching proxy ...
🔍 Verifying proxy health ...
http://127.0.0.1:38347/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/
```

Workloads of Pods

In Kubernetes, **workloads** are the components that manage and deploy applications. Pods are the basic unit of workload in Kubernetes, and they can host one or more containers that run applications.



Pods and Services Metadata

In Kubernetes, both Pods and Services have associated metadata that provides essential information about them. This metadata helps in managing and identifying resources within the cluster.

The screenshot shows the Kubernetes dashboard interface. The left sidebar contains navigation links for Workloads, Cron Jobs, Daemon Sets, Deployments, Jobs, Pods, Replica Sets, Replication Controllers, Stateful Sets, Service, Ingresses, Ingress Classes, Services, Config and Storage, Config Maps, and Persistent Volume Claims. The main content area is divided into three sections: Deployments, Pods, and Replica Sets.

Deployments

Name	Images	Labels	Pods	Created
portfolio	jayasurya5454/jayasuryaportfolio:latest	app: portfolio	2 / 2	34 minutes ago

Pods

Name	Images	Labels	Node	Status	Restarts	CPU Usage (cores)	Memory Usage (bytes)	Created
portfolio-5ff6cb87c-qjbt4	jayasurya5454/jayasuryaportfolio:latest	app: portfolio pod-template-hash: 5ff6cb87c	minikube	Running	0	-	-	34 minutes ago
portfolio-5ff6cb87c-qh8b9	jayasurya5454/jayasuryaportfolio:latest	app: portfolio pod-template-hash: 5ff6cb87c	minikube	Running	0	-	-	34 minutes ago

Replica Sets

Name	Images	Labels	Pods	Created
portfolio-5ff6cb87c	jayasurya5454/jayasuryaportfolio:latest	app: portfolio pod-template-hash: 5ff6cb87c	2 / 2	34 minutes ago

The screenshot shows the Kubernetes dashboard interface with the 'Service' tab selected. The left sidebar contains navigation links for Pods, Replica Sets, Replication Controllers, Stateful Sets, Service, Ingresses, Ingress Classes, Services, Config and Storage, Config Maps, Persistent Volume Claims, Secrets, Storage Classes, Cluster, Cluster Role Bindings, Cluster Roles, and Events.

Services

Name	Labels	Type	Cluster IP	Internal Endpoints	External Endpoints	Created
portfolio-service	-	NodePort	10.107.93.246	portfolio-service:8080 TCP portfolio-service:32437 TCP	-	39 minutes ago
kubernetes	component: apiserver provider: kubernetes	ClusterIP	10.96.0.1	kubernetes:443 TCP kubernetes:0 TCP	-	23 days ago

